Eric Zheng

Professor Steven Hulse

October 29th, 2026

CISC 3130

Supplementary Project Document

A) Big–O Table

Method	Adjacent Structures	Big-O	Reason
addSong	Hashmap, Doubly linked list	O(1)	Hashmap lookup is constant time and appending to the linked list is constant time.
findSong	Hashmap	O(1)	Hashmap lookup is constant time.
play	Hashmap, Stack	O(1)	Hashmap lookup is constant time and pushing to the stack is constant time.
queueLater	Hashmap, Doubly linked list	O(1)	Hashmap lookup is constant time and appending to the ends of the doubly linked list is constant time.
queueNext	Hashmap, Doubly linked list	O(1)	Similar to queueLater, hashmap lookup is constant time and appending to the ends of the doubly linked list is constant time.
next	Doubly linked list, Stack	O(n)	The next method iterates through a doubly linked list, modifies the ends of a doubly linked list, and pushes to a stack, which has time complexities of linear, constant, and constant yielding an overall time complexity of linear time.
back	Doubly linked list, Stack	O(n)	The back method iterates through a doubly linked list and pops from a stack, which has time complexities of linear, and constant, yielding overall time complexity of linear time.

removeSong	Hashmap, Doubly linked list	O(n)	Iterates through a linked list and hashmap lookup with time complexities of linear and constant, yielding overall time complexity of linear time.
showPlaylist	Doubly linked list	O(n)	Iterates through a linked list with a time complexity of linear time.

B) Defining Data structure choices

1b. Stack

- I used a stack as its defining ruleset matches the applications for history. The history stores the recently played song such that the accessibility is in reverse chronological order which makes methods such as back() easy to implement. If the Stack implementation is switched to a standard array, the time complexity will not change as both would be able to push and pop from the history in constant time. The built stack uses nodes with pointers allowing constant time pushing and popping since the memory address of the head node is always stored.

2b. Doubly linked list

- I used a doubly linked list as it provided flexibility with sizing and inserting at any point within the list. The doubly linked list is the backbone of both queue and playlist variables of the music player as it allows for easy removal of elements at any index, constant time pushing and popping from either ends, and a dynamic size ensuring scalability. If the playlist and queue used a standard array instead of a doubly linked list, size limitations will become apparent. In addition, inserting to the front of the list will increase in time complexity from constant time to linear time, while inserting to the back will remain in constant time and popping an element in linear time.